

Design Document: BMTP Motion-Generation Engine (trajectory/+bmtEngine)

Author: Kevin Trinh
Status: Draft
Last updated: 2026-08-31
Reviewers: TBD

Contents

1	Context	2
2	Goals	2
3	Non-goals	2
4	Overview	2
5	Detailed design	4
5.1	Public surface	4
5.2	What the planner provides and what the engine returns	4
5.3	The BMTP solve loop	5
5.4	Certification before acceptance	5
6	Public interface reference	5
6.1	solve	5
6.2	createDirectMotion	6
6.3	createDelayedMotion	7
6.4	createOffsetSplineMotion	7
6.5	createProgressPolynomialMotion	8
6.6	createMotionRecord	8
6.7	createCoordinateTolerances	9
6.8	evaluatePolynomial	9
6.9	maximumRestToRestDistance	10
6.10	convertPolynomialToWarmStart	10
6.11	certifyWarmStartCollisionFree	11
7	Alternatives considered	11
8	Testing, performance, and failure reporting	12
9	Open questions	12

1 Context

The repository provides one public obstacle-avoidance planner in the azimuth/elevation frame. The planner owns obstacle interpretation, topology search, candidate selection, and authoritative validation. It delegates all motion generation to a separate engine, `trajectory/+bmtEngine`.

The engine is dimension-neutral. It does not import obstacle or planner packages. It receives only numeric inputs: a topology seed, finite convex exclusion polygons, normalized endpoint states, and normalized limits. This boundary lets the planner change its geometry pipeline without an engine change, and lets tests exercise the engine in isolation.

Position uses degrees. Time uses seconds. Derivatives use deg/s , deg/s^2 , and deg/s^3 .

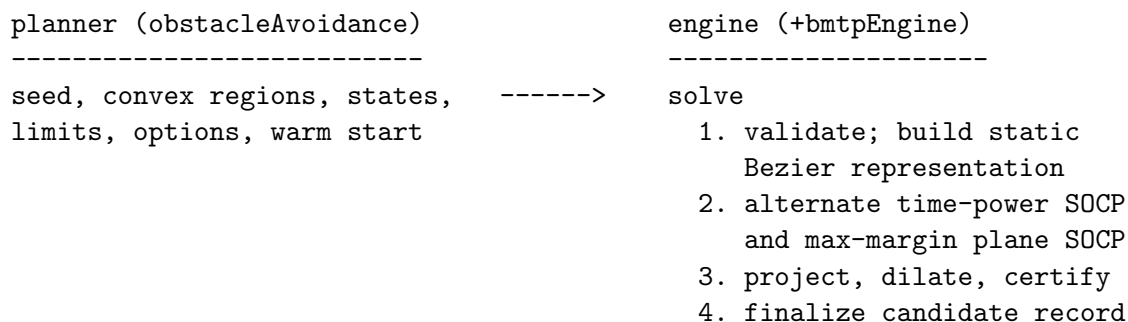
2 Goals

- Generate exact minimum-time synchronized rest-to-rest motion with analytic jerk-switching laws and lossless time scaling.
- Optimize one static topology seed with a composite Bezier trajectory, third-order time-power cones, and alternating degree-one maximum-margin separating planes (the BMTTP solve).
- Accept a motion only after direct Bernstein checks and an all-span, all-region separating-plane certificate.
- Return stable diagnostics for every outcome, including expected infeasibility, work-limit stops, and stagnation stops.
- Provide exact supporting constructions: dwell prepend, quintic coordinate offsets, lateral progress composition, and event-word integration.

3 Non-goals

- Obstacle interpretation. The planner converts protected geometry to numeric convex exclusion regions before any engine call.
- Authoritative acceptance. The public validator remains the final authority for coverage, continuous collision freedom, workspace, endpoints, and derivative limits.
- Global optimality proofs. A validated candidate is a feasible incumbent unless a request-wide physical lower certificate applies.
- Dynamic obstacle families. Cavity, timed-opening, and fixed-clock lateral constructions stay planner-owned because they interpret geometry and timing.

4 Overview



candidate, diagnostics, <-----
 restart (certified net)

Obstacle-free rest-to-rest requests bypass the SOCP path. They use the exact synchronized switching kernel (`createDirectMotion`). Fixed arrival times stretch the same law without increasing any derivative.

Figure 1 replaces the earlier text diagram. A topology seed is a route-shaped point list. A separating plane is a line that puts a curve span and a convex exclusion region on opposite sides.

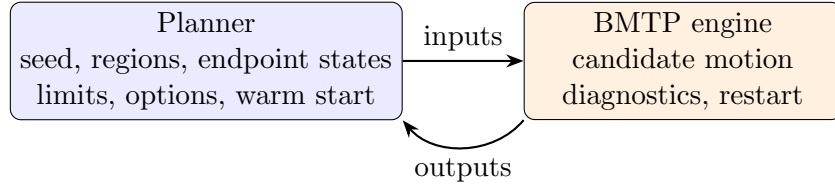


Figure 1: Boundary between obstacle avoidance and the engine.

Figure 2 shows `solve`. The time-power SOCP is a convex program that applies derivative limits while duration is a variable.

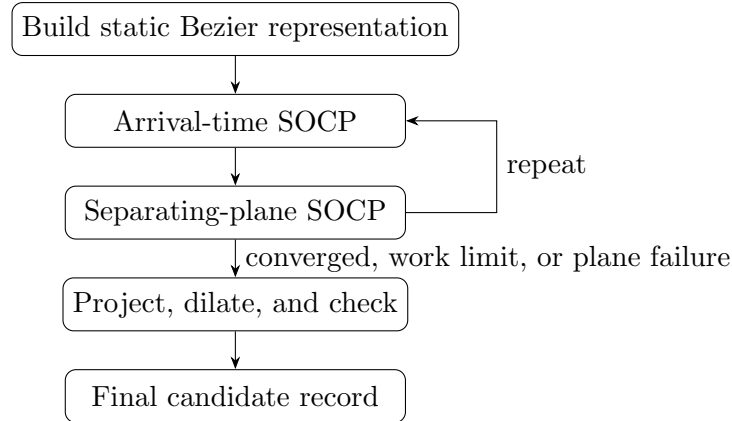


Figure 2: The alternating BMTP solve and its three stopping outcomes.

Figure 3 shows the warm-start path. Bernstein form expresses a polynomial by control points. Its convex-hull property bounds a full span, not only sampled points.

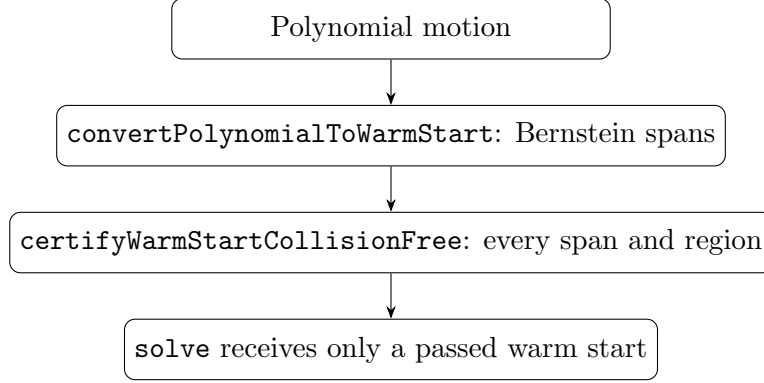


Figure 3: Warm-start approximation and collision certification.

5 Detailed design

5.1 Public surface

Function	Responsibility
<code>solve</code>	Optimize one static seed; return candidate, diagnostics, restart.
<code>createDirectMotion</code>	Exact minimum-time synchronized rest-to-rest motion.
<code>createDelayedMotion</code>	Prepend a stationary dwell; repartition an exact cubic motion.
<code>createOffsetSplineMotion</code>	Add a minimum-integrated-jerk quintic offset to one coordinate.
<code>createProgressPolynomialMotion</code>	Compose an endpoint-flat lateral path over one straight progress motion.
<code>createMotionRecord</code>	Build the shared candidate record; integrate a piecewise-constant-jerk event word exactly.
<code>createCoordinateTolerances</code>	Derive one coordinate scale and the shared geometric tolerances.
<code>evaluatePolynomial</code>	Evaluate ascending-power segment records at absolute times.
<code>maximumRestToRestDistance</code>	Exact maximum scalar displacement in one clock.
<code>convertPolynomialToWarmStart</code>	Approximate a piecewise polynomial with equal-duration Bernstein spans.
<code>certifyWarmStartCollisionFree</code>	Certify every warm-start span against every region before BMTP.

5.2 What the planner provides and what the engine returns

- **Seed.** `position_deg` is N-by-2. `tau` strictly increases from zero to one.
- **Regions.** An R-by-1 cell array. Each cell holds one finite convex N-by-2 exclusion polygon in degrees. Plane witnesses certify only these supplied regions.
- **Coverage.** A planner-owned proof record. The engine requires `Passed`. When the `Applied` field in `ConservativeGrouping` is true, the solve uses the compact degree-7 representation with one split; otherwise it uses the default representation.
- **Warm start.** A previously certified `ControlPoint_deg` array and `SegmentTime.s`. `certifyWarmStartCollisionFree` must pass before entering the SOCP loop.
- **Candidate.** One stable motion record shared by all construction paths. Expected infeasibility returns `Success=false` with preserved diagnostics.
- **Restart.** The certified parent control net and segment time, returned for exact later refinement.

5.3 The BMTP solve loop

`solve` alternates two convex programs over a composite Bezier curve:

1. **Time-power SOCP.** Third-order time-power cones minimize arrival time subject to derivative limits, endpoint conditions, and the current separating planes.
2. **Maximum-margin plane SOCP.** Degree-one Bernstein separating planes are re-derived for every span/region pair to maximize clearance margin.

The loop stops on convergence, on the work limit, or on the stagnation stop. The immutable conic structure (derivative, endpoint, and continuity matrices; equality right-hand side; objective; bound template; second-order cones) is cached per seed. Plane-dependent rows and the active horizon bound are rebuilt on every alternation in the original order.

5.4 Certification before acceptance

Acceptance requires all of the following, in order:

1. Direct Bernstein checks on the selected motion. Segment-local power coefficients are converted to Bernstein form. The convex-hull property bounds complete polynomial intervals; the engine does not rely on sampled points alone.
2. An all-span, all-region degree-one plane certificate after projection and dilation.
3. Independent public validation by the planner-side validator. The engine never self-certifies a final result.

Figure 4 shows the geometric test. Clearance is the certified distance between the span and the region.

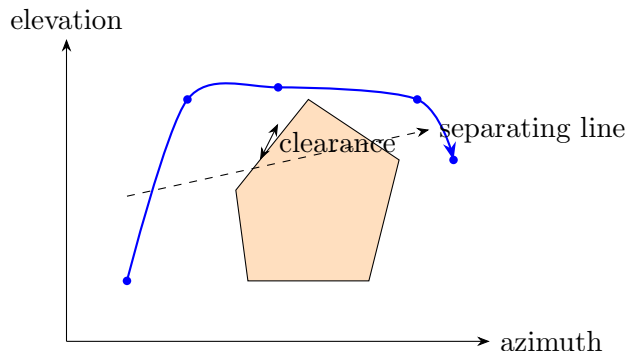


Figure 4: A polygon, a composite Bezier path, and a separating line.

Figure 5 relates the exact motion builders.

6 Public interface reference

6.1 `solve`

Optimizes one static topology seed. The planner calls it after it has created convex exclusion regions and normalized the request.

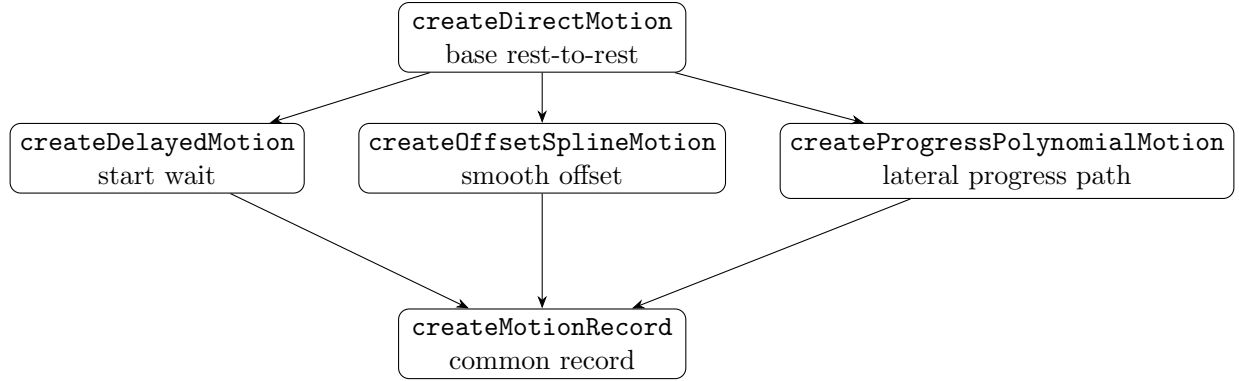


Figure 5: How exact motion builders create a common candidate record.

```
[candidate, diagnostics, restart] = bmtEngine.solve(seed, regions_deg,
coverage, initialState, goalState, limits, options, warmStart)
```

Name	Shape/type	Meaning	Units
seed	scalar struct	N-by-2 route and increasing tau	degrees
regions_deg	R-by-1 cell array	convex exclusion polygons	degrees
coverage	scalar struct	planner coverage proof with Passed	–
initialState,	scalar structs	rest endpoints and times	degrees,
goalState			seconds
limits, options	scalar structs	bounds, time policy, work limits	mixed
warmStart	scalar struct or empty	certified control net and span time	degrees, seconds
Output	Meaning		
candidate	stable motion record		
diagnostics	solver, timing, coverage, and certificate evidence		
restart	parent control net and segment time		

Invalid inputs and malformed warm starts raise errors. Expected infeasibility returns `Success=false`, including no feasible iterate, an infeasible time window, or an unavailable plane certificate.

6.2 createDirectMotion

Creates exact synchronized rest-to-rest motion. The planner calls it for a direct request and as a base for later composition.

```
candidate = bmtEngine.createDirectMotion(initialState, goalState, limits,
options)
```

Name	Shape/type	Meaning	Units
initialState, goalState	scalar structs	time, position, optional rest derivatives	degrees, seconds
limits	scalar struct	positive velocity, acceleration, jerk bounds	derivative units
options	scalar struct	goal-time mode, sample time, tolerance	seconds, de- grees

Output	Meaning
candidate	stable exact polynomial and sampled-motion record

Invalid states, limits, options, and time horizons raise errors. Unsupported endpoint derivatives, zero displacement, too-short arrival, and reconstruction failure return **Success=false**.

6.3 createDelayedMotion

Adds a stationary dwell before an existing cubic motion. The planner calls it when a wait must keep the exact base motion.

```
candidate = bmtEngine.createDelayedMotion(baseMotion, initialState,
breakTime_s, sampleStep_s, seedSource)
```

Name	Shape/type	Meaning	Units
baseMotion	scalar result struct	complete cubic polynomial	–
initialState	scalar struct	start state	degrees, seconds
breakTime_s	numeric vector	increasing absolute break times	seconds
sampleStep_s, seedSource	scalar, scalar text	history spacing and source label	seconds, –

Output	Meaning
candidate	exact delayed polynomial and histories

Invalid calls, degree, partition, sampling, and labels raise errors. A valid request has no separate normal failure record.

6.4 createOffsetSplineMotion

Adds a smooth scalar offset to one coordinate. The planner calls it when offset knots define a lateral change. It uses a minimum-integrated-jerk quintic spline, which minimizes the integral of jerk squared.

```
candidate = bmtEngine.createOffsetSplineMotion(baseMotion, knotTime_s,
knotOffset_deg, axisIndex, initialState, sampleStep_s, seedSource)
```

Name	Shape/type	Meaning	Units
<code>baseMotion</code>	scalar result struct	complete base polynomial	–
<code>knotTime_s,</code> <code>knotOffset_deg</code>	matching vectors	increasing offset knots	seconds, de- grees
<code>axisIndex</code>	positive integer	offset coordinate	–
<code>initialState,</code> <code>sampleStep_s,</code> <code>seedSource</code>	struct, scalar, text	state, spacing, label	mixed

Output	Meaning
<code>candidate</code>	composite polynomial and sampled histories

Invalid calls, knots, axes, clock ends, sampling, and labels raise errors. A valid construction returns a motion record.

6.5 createProgressPolynomialMotion

Layers an endpoint-flat lateral path over a straight-progress motion. Endpoint-flat means the added path and its endpoint derivatives vanish.

```
candidate = bmtEngine.createProgressPolynomialMotion(baseMotion, initialState,
goalState, progressAxisIndex, amplitude_deg, sampleStep_s, seedSource)
```

Name	Shape/type	Meaning	Units
<code>baseMotion</code>	scalar result struct	successful two-coordinate straight progress	–
<code>initialState,</code> <code>goalState</code>	scalar structs	two-coordinate rest endpoints	degrees, seconds
<code>progressAxisIndex,</code> <code>amplitude_deg</code>	integer, scalar	progress coordinate and lateral co- efficient	–, degrees
<code>sampleStep_s,</code> <code>seedSource</code>	scalar, text	spacing and label	seconds, –

Output	Meaning
<code>candidate</code>	exact degree-15 composed motion

Invalid base motion, endpoints, axes, zero progress, or labels raise errors. A valid construction returns a motion record.

6.6 createMotionRecord

Creates the common candidate record. Builders call it to integrate an event word exactly. An event word is time breaks with constant jerk per interval. It can also package an assembled polynomial.

```
[candidate, terminalState] = bmtEngine.createMotionRecord(candidate,
initialState, relativeBreak_s, segmentJerk_deg_s3, sampleStep_s, seedSource)
```


Name	Shape/type	Meaning	Units
<code>candidate</code>	scalar struct	empty creates the common record	–
<code>initialState</code>	scalar struct	position, velocity, acceleration	derivative units
<code>relativeBreak_s</code>	vector, polynomial, or empty	event breaks or polynomial	seconds
<code>segmentJerk_deg_s3</code>	N-by-D numeric	constant interval jerk	degrees per second cubed
<code>sampleStep_s,</code> <code>seedSource</code>	scalar, text	spacing and label	seconds, –

Output	Meaning
<code>candidate</code>	stable polynomial and history record
<code>terminalState</code>	exact terminal state

Empty breaks return the stable not-run record. Invalid polynomials and event words raise errors.

6.7 createCoordinateTolerances

Derives one coordinate scale and shared geometric tolerances. The engine uses it where construction and collision checks need the same roundoff allowance. A roundoff reserve is an allowance for finite precision.

```
[coordinateScale_deg, geometryTolerance_deg, roundoffReserve_deg] =
bmtEngine.createCoordinateTolerances(values_deg, ...)
```

Name	Shape/type	Meaning	Units
<code>values_deg</code>	numeric arrays or cells	coordinate collections; values ignored	nonfinite degrees

Output	Meaning
<code>coordinateScale_deg</code>	maximum finite absolute coordinate, at least one degree
<code>geometryTolerance_deg</code>	shared geometric tolerance
<code>roundoffReserve_deg</code>	conservative finite-precision reserve

Nonnumeric coordinate collections raise errors. Valid calls always return tolerances and have no failure record.

6.8 evaluatePolynomial

Evaluates common ascending-power polynomial records at absolute times. Ascending power means coefficients start with the constant term. The engine uses it for histories and warm-start comparisons.

```
[time_s, position_deg, velocity_deg_s, acceleration_deg_s2, jerk_deg_s3] =
bmtEngine.evaluatePolynomial(polynomial, time_s, segmentIndex)
```

Name	Shape/type	Meaning	Units
<code>polynomial</code>	scalar struct	N-by-D-by-P ascending-power records	derivative units
<code>time_s</code>	numeric vector	absolute request times	seconds
<code>segmentIndex</code>	scalar or vector, optional	exact segment selection	–

Output	Meaning
<code>time_s</code>	normalized N-by-1 time column
<code>position_deg</code>	through N-by-D histories
<code>jerk_deg_s3</code>	

Empty or nonfinite times return NaN histories. Malformed polynomial data and incompatible indices raise ordinary MATLAB errors.

6.9 `maximumRestToRestDistance`

Returns the exact maximum scalar rest-to-rest displacement for one clock. It uses the same jerk-limited, acceleration-limited, and velocity-limited switching regimes as the direct builder.

```
distance_deg = bmtEngine.maximumRestToRestDistance(duration_s,
velocityLimit_deg_s, accelerationLimit_deg_s2, jerkLimit_deg_s3)
```

Name	Shape/type	Meaning	Units
<code>duration_s</code>	nonnegative scalar	available motion duration	seconds
<code>velocityLimit_deg_s</code>	positive scalar	symmetric velocity limit	degrees per second
<code>accelerationLimit_deg_s2</code>	positive scalar	symmetric acceleration limit	degrees per second squared
<code>jerkLimit_deg_s3</code>	positive scalar	symmetric jerk limit	degrees per second cubed

Output	Meaning
<code>distance_deg</code>	exact maximum displacement

Invalid duration or limits raise errors. Valid calls return a nonnegative distance.

6.10 `convertPolynomialToWarmStart`

Approximates a piecewise polynomial by equal-duration Bernstein spans. The planner calls it before warm-start certification. It measures error but does not approve the approximation.

```
[warmStart, diagnostics] = bmtEngine.convertPolynomialToWarmStart(polynomial,
segmentCount, degree)
```

Name	Shape/type	Meaning	Units
<code>polynomial</code>	scalar struct	source segments with local powers	degrees, seconds
<code>segmentCount</code>	positive integer	output span count	–
<code>degree</code>	positive integer	fitted Bernstein degree	–
Output	Meaning		
<code>warmStart</code>	control points and common span time		
<code>diagnostics</code>	fit and check counts and sampled deviations		
Invalid polynomial records, span counts, and degrees raise errors. A valid conversion returns an approximation even if its measured error is large.			

6.11 `certifyWarmStartCollisionFree`

Checks every Bernstein warm-start span against every exclusion region. It tries a constant separator, then the degree-one separating-plane form. The planner calls it before it passes a warm start to `solve`.

```
certificate = bmtEngine.certifyWarmStartCollisionFree(warmStart, regions_deg,
collisionClearanceTolerance_deg)
```

Name	Shape/type	Meaning	Units
warmStart	scalar struct	control points and positive span time	degrees, seconds
regions_deg	cell vector	finite convex protected polygons	degrees
collisionClearance	nonnegative scalar	required separation after reserve	degrees
Tolerance_deg			
Output	Meaning		
certificate	all-pair result, planes, reasons, counts, minimum clearance		
Invalid inputs raise errors. An uncertified warm start returns a failure record with Passed=false and reason uncertifiedWarmStartCollision.			

7 Alternatives considered

- **Rebuild the full SOCP every alternation (previous behavior).** Rejected for cost. Profiling showed the immutable conic structure was rebuilt unchanged. The retained cache improved the maintained Target Exits median wall time by 9.82% and the diagnostic mode by 15.34%, with bit-identical reference example results (see `verification.md`).
- **Separating-plane reuse across alternations.** Rejected. The experiment changed accepted results without a controlling bound. The restored version is off unless the planner sets `EnablePlaneReuse` to true, with a retained-best improvement bound; the default path preserves the plane reset and re-derivation exactly (see `branch_assessment.md`).
- **Sample-based collision checking.** Rejected. Sampling cannot bound a full polynomial interval. Bernstein convex-hull bounds certify complete spans.
- **Engine-side obstacle interpretation.** Rejected. It would couple the engine to planner geometry and break dimension neutrality.

8 Testing, performance, and failure reporting

- **Testing.** The clean MATLAB suite checks every change (currently 98/98). Default reference example cases must remain bit-identical for any change that claims no behavior difference. Changed files must be Code Analyzer clean.
- **Performance.** `coneprog` calls dominate solve cost. Timing claims use isolated MATLAB preference snapshots, one discarded warm-up, and interleaved A/B repetitions.
- **Failure visibility.** Work limits, exhausted alternation, unsupported requests, and physical infeasibility return stable failure records. The engine does not throw for expected infeasibility.

Figure 6 plots measured median wall times from the dated BMTP immutable SOCP cache section of `verification.md`. The values are measured baseline and candidate medians for the four timed cases.

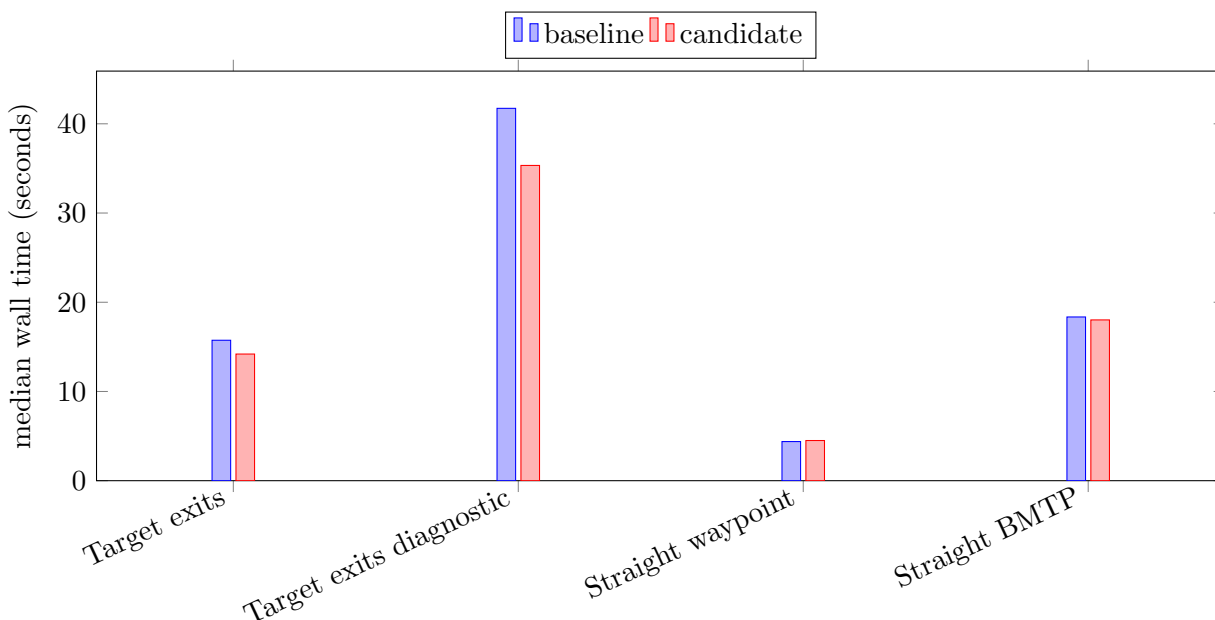


Figure 6: Measured median wall times from `verification.md`. Each pair compares the recorded baseline and candidate medians for one timed case.

9 Open questions

- Should the stagnation stop and `EnablePlaneReuse` controls merge into one alternation-policy option?
- Can the warm-start position-error bound from `convertPolynomialToWarmStart` feed the dilation margin directly, instead of a fixed tolerance?
- Is a request-wide lower-bound certificate practical for seeded BMTP solves, so more results can claim bounded arrival instead of incumbent status?